

Sprint 1

Dylan Shaffer, DylanShaffer412, dylanshaffer1@gmail.com, NASA, Dev-Env-DS Branch

What you planned to do

- Create SQL table to hold solar flare data from API
- Create Python scripts to pull data from API and load into table
- Create web interface to display the data

What you did not do

- Create working web interface

Issues completed

- Create Postgres FLR table #11
- Create FLR interface #5

Files you worked on

- NASA-Data-Viewer/backend/FLR/FLR_Load.py
- NASA-Data-Viewer/backend/FLR/FLR_Transform.py
- NASA-Data-Viewer/backend/FLR/FLR_table.sql
- NASA-Data-Viewer/backend/FLR/api.py
- NASA-Data-Viewer/backend/FLR/main.py
- NASA-Data-Viewer/frontend/SolareFlare.jsx

Use of AI and/or 3rd party software

AI was used in the development of code in this sprint. I do not have a strong background in Python, so I made some very rough scripts that I then fed to Claude for polishing. I did take the time to look into and understand as much as I could any code that was added by Claude. Any third party software used consists of Python libraries: psycopg2 and fastapi. Any other library used is already included in Python (os, dotenv, etc.).

What you accomplished

The overall workflow consisted of the following: Configure the docker container on my machine (help provided by Aaron), setup Postgres in the container on my machine (help provided by Tanner), attempt to configure Vite (unsuccessful in this, files provided by Lane to run Vite but ran out of time before getting a working configuration), then create my table within Postgres, create the Python scripts to pull API data, and create a frontend to display them.

Once my environment was configured, barring Vite, I created the SQL script with no issue and got it loaded into Postgres. I then moved onto the Python scripts. Reading through online documentation and old forum/StackOverflow posts, I pieced together something of script.

I then fed it to Claude and asked it for polishing and dealing with a few errors that had occurred. It did work, but it was a monster script that was very poorly optimized. I met with Tanner on another night, and he walked me through the process he had for his API call. I modified my piece to mimic his for optimization, program structure, and overall cleanliness. I will stress that this is something that I took the time to understand and did not blindly copy.

Once the API was working and could load data into the Postgres database, I started working on the frontend. Due to time constraints, I was only able to get a very basic one that displays the table, and could not test due to Vite not being configured correctly.

Overall, I severely underestimated the difficulty of my sprint items and the time it would take to complete them. My goals for the next sprint are to schedule specific times to work on items, do not procrastinate, and more effectively communicate with my team when it comes to architectural decisions, as well as general layout.